ECEN 4/5632 Theory and Application of Digital Filters
Lecture 22, 11-03-26, 11-10-26
Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

Midterm Exam 2

Thursday November 5, 9:30 – 10:45 am, ECCR 131 for in-class students.
Between Nov. 5 and Nov. 11 for on-line students who must supply email address of exam proctor.
On-line students are welcome to take the exam in-class on Nov. 5, 9:30 – 10:45 am.

Covers (mostly) Lectures 7 through 14 and Problem Sets 5 through 9.
Topics: FIR, IIR filters, windowing, bilinear transformation, time/frequency responses, (generalized) linear phase, group delay, ML intersections (conv1d), DFT/FFT, fast convolution, STFT, spectrograms, downsampling, decimation, CNN strides, upsampling, interpolation, rational rate changes.
Exam problems will be similar to Problem Set questions.

Closed book, notes, no cheat sheets. Reference formulas will be printed on exam.
No laptops, tablets, phones, basic (non-programmable) scientific calculator o.k.

WEEK 11 ● LECTURE 22

Efficient Interpolation and Filter Banks

Polyphase synthesis, perfect reconstruction, and fixed/learned subband front ends

↑L + polyphase = no zero multiplies

analysis → subbands → synthesis

Oppenheim & Schafer §4.7–4.8 | Raschka: convolutional architectures / efficient ML

Roadmap and learning objectives

75 minutes • efficient interpolation → two-channel banks → ML subband front ends

for upsampling

- Use the Second Noble Identity correctly in an interpolator.
- Explain why polyphase interpolation saves about a factor L in arithmetic.
- Describe analysis, decimation, synthesis, and perfect reconstruction in a two-channel bank.
- Relate fixed DSP filter banks to compact feature extractors and learned front ends.

0–35 min

Efficient interpolation + Noble identity

35–55 min

Two-channel analysis / synthesis banks

55–75 min

Fixed and learned filter-bank features

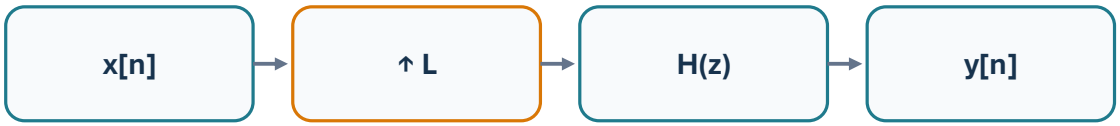
01

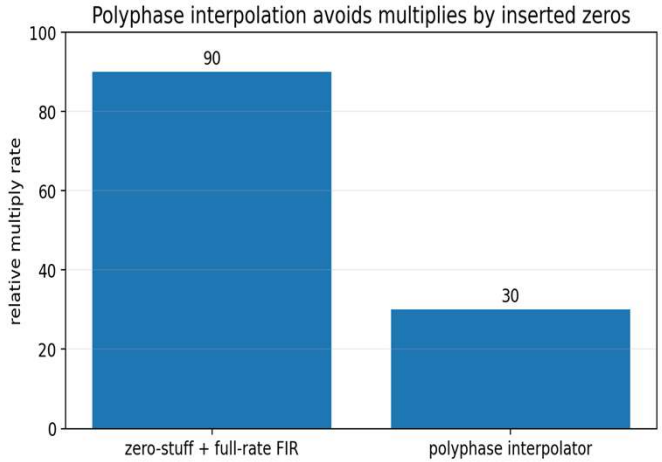
Efficient interpolation

Compute low-rate branch outputs first, then interleave them onto the high-rate grid.

Why naive interpolation wastes arithmetic

Zero insertion creates L−1 zeros between input samples, then a full-rate FIR multiplies them anyway.





Naive viewpoint

The FIR runs at $L \cdot F_s$ even though most values entering it are known zeros.

Polyphase viewpoint

Each of the L output phases is a low-rate convolution of x[n] with one phase $E_k(z)$.

$$H(z) = \Sigma_{k=0}^{L-1} z^{-k} E_k(z^L)$$

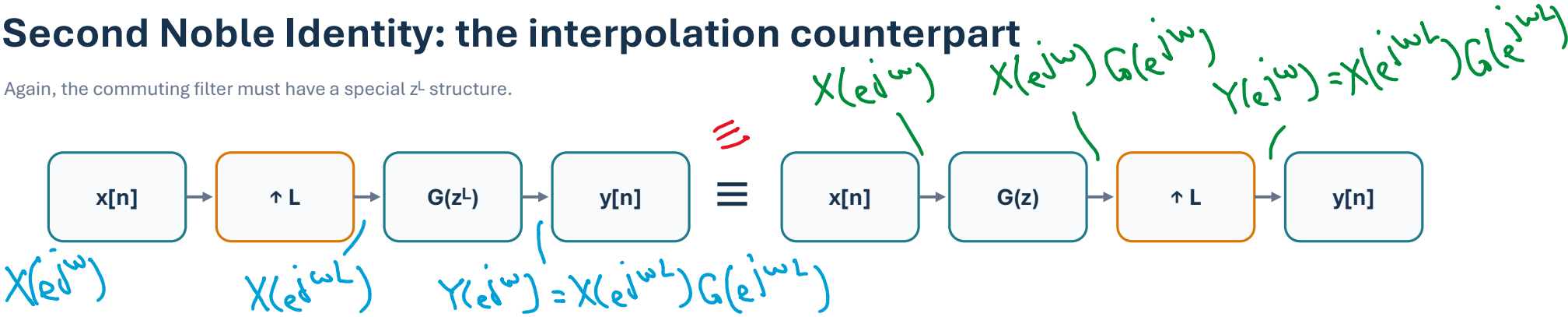
$E_k(z) = \sum_n e_k[n] z^{-n}$

$e_k[n] = h[Ln + k]$

$H(z) = \sum_{k=0}^{L-1} E_k(z^L) z^{-k}$

Second Noble Identity: the interpolation counterpart

Again, the commuting filter must have a special z^L structure.



$$\uparrow L \rightarrow G(z^L) \equiv G(z) \rightarrow \uparrow L$$

Efficiency

On the right, G(z) executes at the lower input rate.

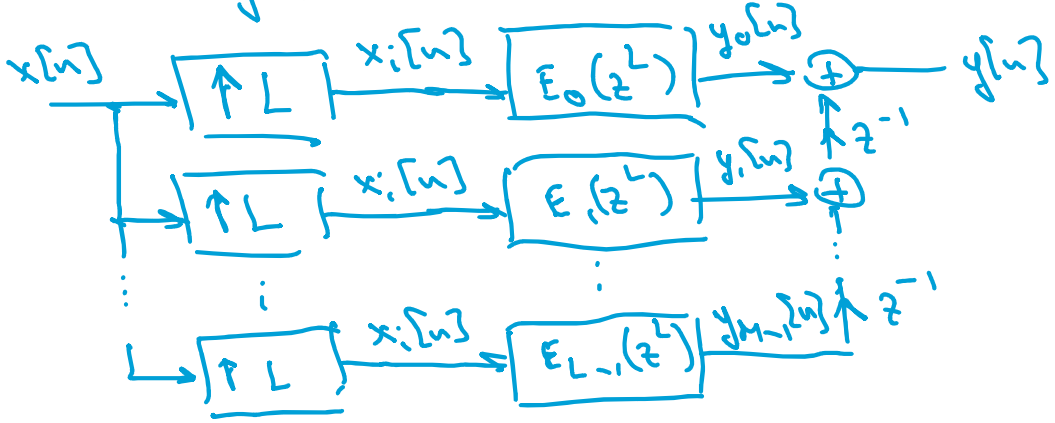
Caution

An arbitrary G(z) cannot be commuted across ↑L.

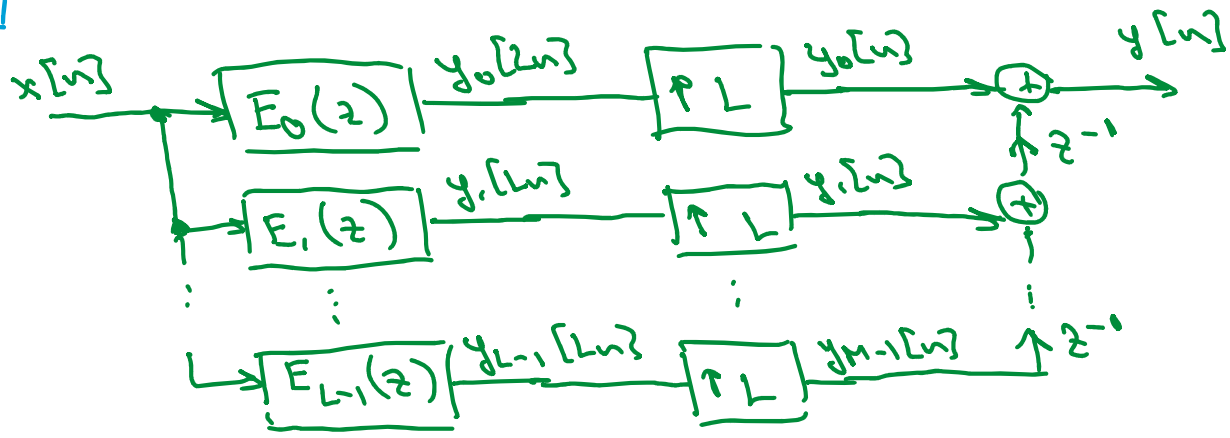
Polyphase Upsampling

$$Y(z) = X(z^L) \cdot H(z) = \sum_{k=0}^{L-1} X(z^L) E_k(z^L) z^{-k}$$

naive way first



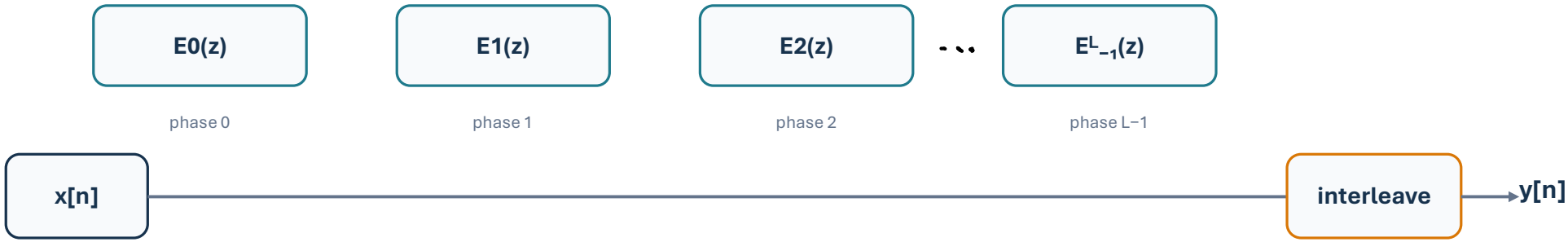
efficient way (using Noble identity)



filter first, upsample later ⇒ savings ≈ L

Polyphase interpolator: L low-rate filters generate L output phases

Filter first at F_s, then a commutator interleaves branch samples.



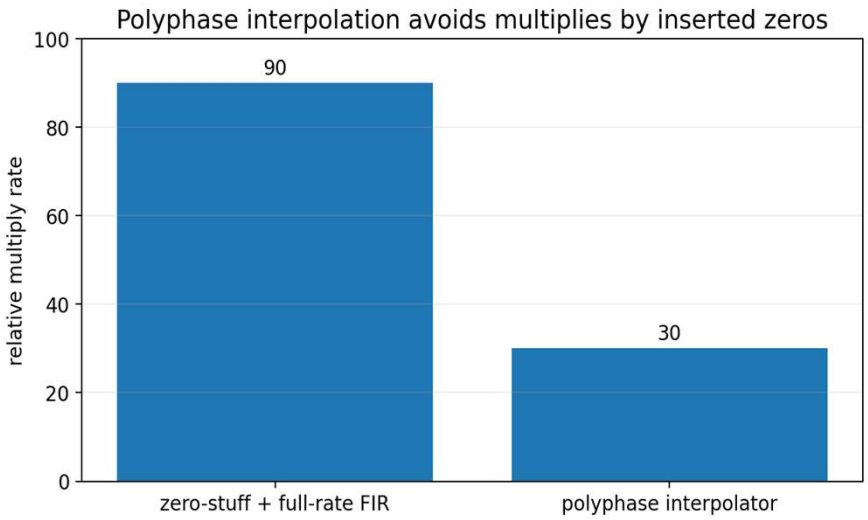
$$\text{output phase } k\text{: } y[mL+k] = (e_k * x)[m]$$

Key idea

No branch ever multiplies an inserted zero. Each branch computes only the samples assigned to its output phase.

L-fold savings and exact equivalence

Polyphase changes the schedule, not the interpolation result.



L = 3 example

- Naive: roughly $N \cdot L \cdot F_s$ coefficient multiplies/s for an N-tap full-rate FIR.
- Polyphase: roughly $N \cdot F_s$ multiplies/s total across all phases.
- Ideal steady-state saving ≈ L.
- scipy.signal.upfirdn implements this multirate idea efficiently.

Amplitude reminder

Efficiency does not replace interpolation design: the anti-imaging filter still needs the appropriate cutoff and gain L.

CONCEPT CHECK

A 36-tap FIR interpolates by L = 4. Compared with a naive zero-stuff + full-rate FIR implementation, what ideal arithmetic saving should polyphase give?

CONCEPT CHECK

A 36-tap FIR interpolates by L = 4. Compared with a naive zero-stuff + full-rate FIR implementation, what ideal arithmetic saving should polyphase give?

Good answer

Approximately a factor of 4. The polyphase branches collectively perform about $36 \cdot F_s$ multiplies/s instead of about $36 \cdot 4 \cdot F_s$.

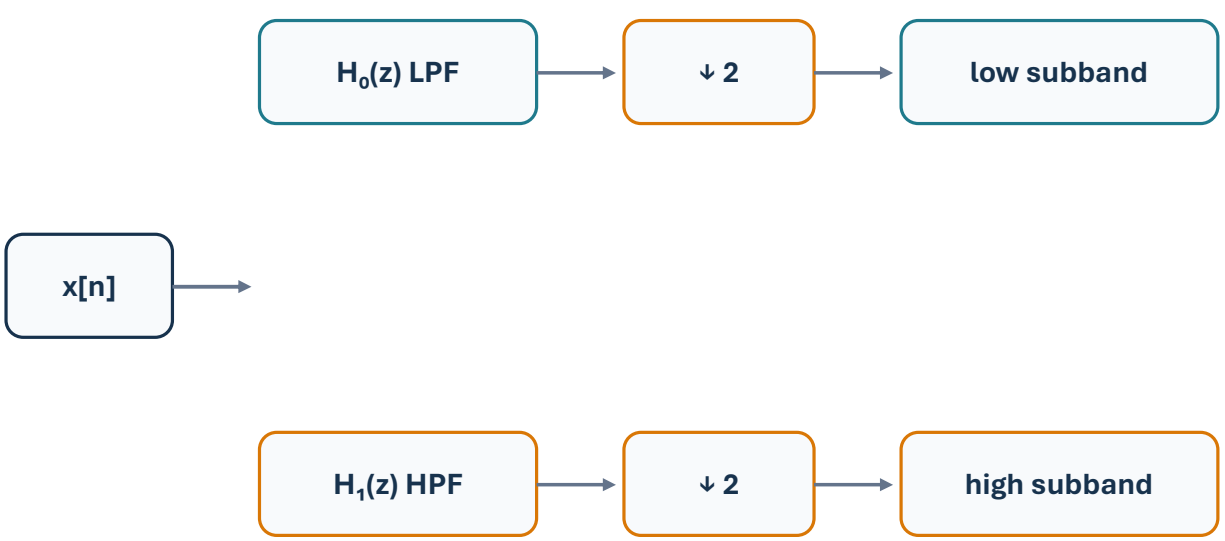
02

Two-channel filter banks

Split the signal into subbands, reduce each rate, then combine them again.

Analysis bank: separate frequency content, then decimate

Each subband occupies less bandwidth, so each branch can often be represented at a lower sample rate.



Critical sampling

Two branches at half-rate contain the same total sample rate as the original.

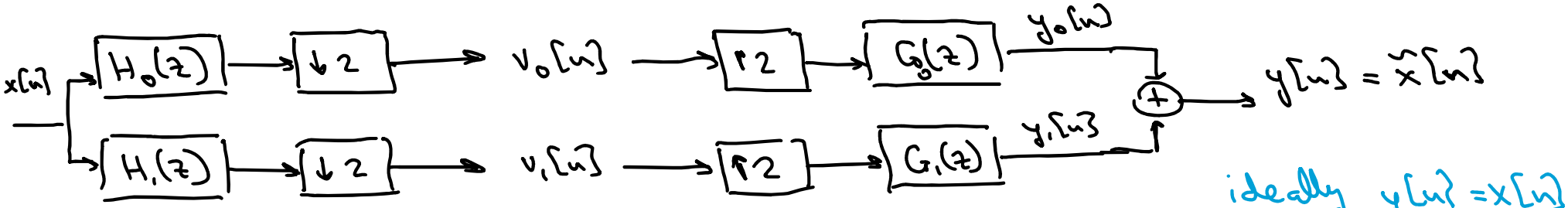
Why filtering first?

As in decimation, each branch needs bandlimiting before ↓2.

Uses

Compression, coding, denoising, subband features.

2 channel filterbank



ideally $y[n] = x[n]$

ideally:

$H_0(z)$ is ideal LPF $\omega_c = \pi/2$

$H_1(z)$ is ideal HPF $\omega_c = \pi/2$

ideally

$G_0(z)$ is ideal LPF $\omega_c = \pi/2$

$G_1(z)$ is ideal HPF $\omega_c = \pi/2$

Conditions for perfect reconstruction

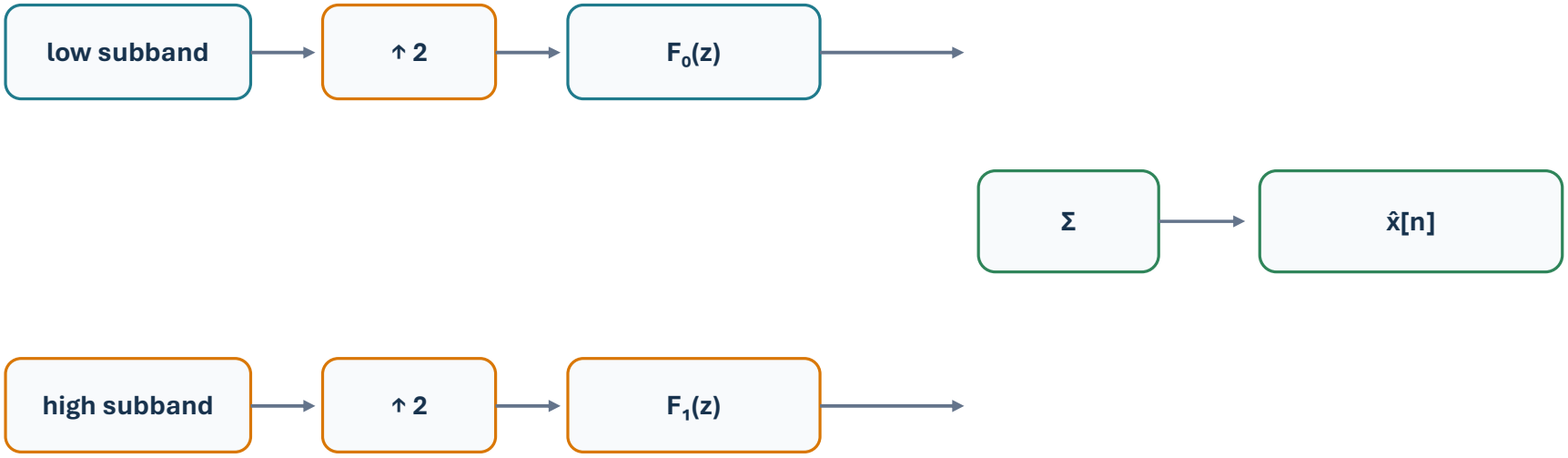
$$G_0(z)H_0(-z) + G_1(z)H_1(-z) = 0$$ Alias cancellation

$$G_0(z)H_0(z) + G_1(z)\cdot H_1(z) = c\, z^{-m}$$ No distortion

(c: constant, z^{-m}: pure delay)

Synthesis bank: upsample, filter, and add

The synthesis filters cancel alias components and restore the desired signal response.



perfect reconstruction: $\hat{x}[n] = c \cdot x[n-d]$

Perfect reconstruction has two simultaneous requirements

One condition cancels aliasing; the other preserves the desired signal path.

$$\text{alias term: } F_0(z)H_0(-z) + F_1(z)H_1(-z) = 0$$

$$\text{distortion term: } \tfrac{1}{2}[F_0(z)H_0(z)+F_1(z)H_1(z)] = c\, z^{-d}$$

Alias cancellation

Downsampling creates spectral folding in each branch; synthesis combines branches so those alias terms cancel.

Desired path

The remaining transfer function should be only a scale and delay—not frequency-dependent distortion.

Mixed-level strategy

First-year students can understand "split → downsample → upsample → recombine"; seniors can use the z-domain conditions above.

Haar bank: the simplest perfect-reconstruction example

Pairs become an average-like lowpass coefficient and a difference-like highpass coefficient.

$v_0[n]$

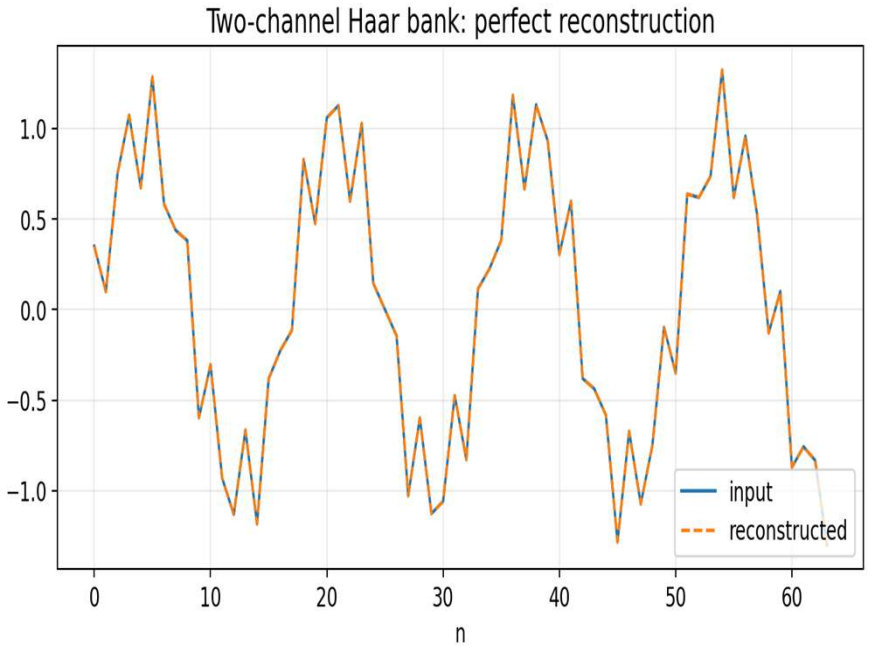
$$a[m] = (x[2m] + x[2m+1]) / \sqrt{2}$$

$v_1[n]$

$$d[m] = (x[2m] - x[2m+1]) / \sqrt{2}$$

$$x[2m] = (a[m]+d[m]) / \sqrt{2}$$

$$x[2m+1] = (a[m]-d[m]) / \sqrt{2}$$



$H_0(z) = \frac{1}{\sqrt{2}}(1+z^{-1})$ LPF like

$H_1(z) = \frac{1}{\sqrt{2}}(1-z^{-1})$ HPF like

$G_0(z) = \frac{1}{\sqrt{2}}(1+z)$

$G_1(z) = \frac{1}{\sqrt{2}}(1-z)$

Haar filter bank

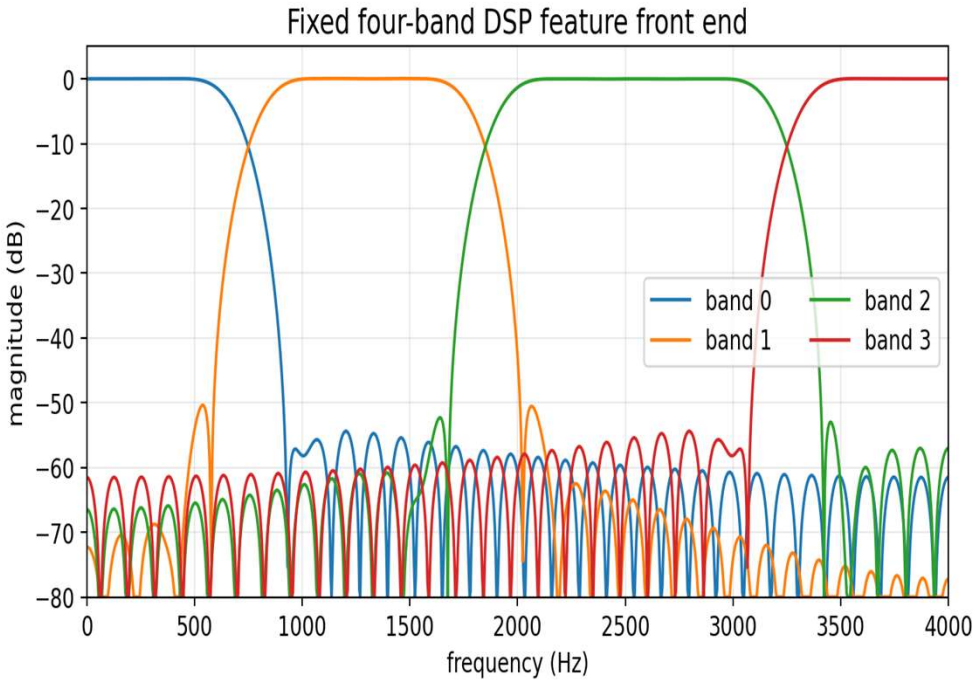
03

ML context: fixed and learned filter banks

Subband structure can be engineered, frozen, or learned—depending on data, compute, and robustness needs.

Fixed subband features can be an efficient front end

Filter → summarize each band → feed a small classifier.



Examples

Mel filter-bank energies, wavelet coefficients, scattering-style fixed transforms, hand-designed subbands.

Why useful

Few trainable parameters, predictable spectral behavior, potentially lower data and compute requirements.

Tradeoff

A fixed front end cannot adapt its filters to every task; useful information may be discarded.

Learned filter banks: let the data choose the subbands

Conv1d kernels can act as a trainable analysis bank, optionally with stride / pooling for rate reduction.

$$\text{feature channel } k:\ y_k[n] = (h_k * x)[n]$$

Connection to Week 8

Any stride/downsampling in the learned analysis bank still raises the anti-aliasing questions from multirate DSP.

Connection to polyphase

Efficient implementations exploit structure to avoid dense redundant computation.

- Fixed bank: interpretable, cheap, stable prior.
- Learned bank: task-adaptive but needs data, training, and regularization.
- Hybrid: initialize from DSP filters, then fine-tune.
- Edge pattern: fixed or depthwise front end + small pointwise / linear classifier.

Simple ML Processing of Filterbank Output

```
# Logistic regression using Scikit-Learn
X = np.asarray(X); y = np.asarray(y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=4, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(Xtr, ytr)
pred = model.predict(Xte)
print("Fixed filter-bank feature accuracy =", accuracy_score(yte, pred))
print("confusion matrix:\n", confusion_matrix(yte, pred))
print("Only 4 fixed subband-energy features feed the classifier.")
```

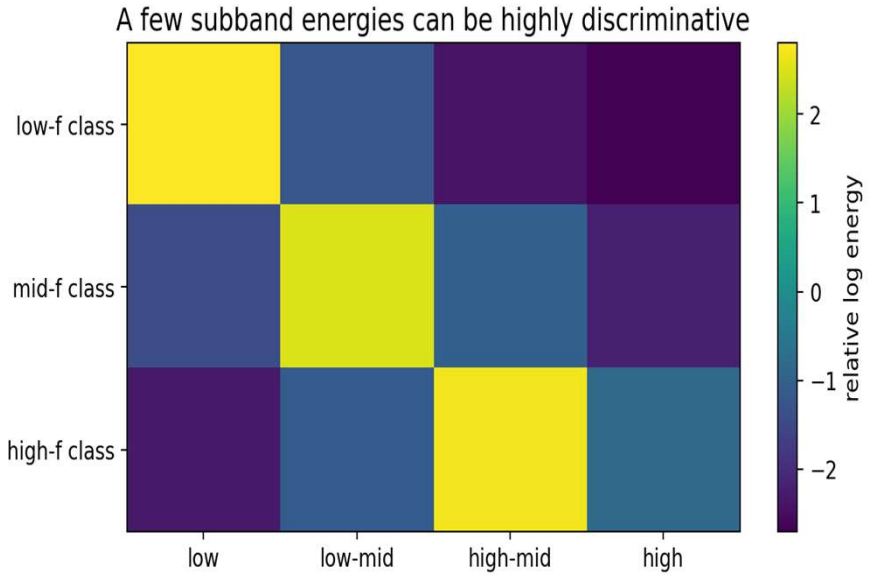
```
Fixed filter-bank feature accuracy = 0.7283950617283951
confusion matrix:
 [[40 14  0]
 [15 31  8]
 [ 0  7 47]]
Only 4 fixed subband-energy features feed the classifier.
```

```
# Tiny PyTorch multiclass logistic regression model
Xt = torch.tensor(np.asarray(X), dtype=torch.float32)
yt = torch.tensor(y, dtype=torch.long)
model = torch.nn.Linear(4, 3)
opt = torch.optim.Adam(model.parameters(), lr=0.04)
for _ in range(120):
    opt.zero_grad()
    loss = torch.nn.functional.cross_entropy(model(Xt), yt)
    loss.backward(); opt.step()
acc = (model(Xt).argmax(1) == yt).float().mean().item()
print(f"Tiny 4→3 PyTorch classifier training accuracy: {acc:.3f}")
print("DSP front end is fixed; only the small classifier is learned.")
```

```
Tiny 4→3 PyTorch classifier training accuracy: 0.948
DSP front end is fixed; only the small classifier is learned.
```

Scikit-Learn/PyTorch demo: four fixed bands, tiny classifier

A compact DSP feature vector can make a simple classification problem nearly linearly separable.



- Generate noisy low-, mid-, and high-frequency signal classes.
- Apply four fixed FIR bands.
- Use log subband energy as a 4-D feature vector.
- Train logistic regression—or a 4→3 PyTorch linear layer.
- Compare with a learned raw-waveform front end as an extension.

Teaching point

Feature engineering can move complexity from the learned model into a known, efficient signal-processing transform.

Lecture 22 wrap-up

Polyphase is the implementation engine; filter banks turn the idea into parallel signal representations.

- Second Noble Identity moves G(z) to the low-rate side of ↑L when the high-rate form is $G(z^L)$.
- Polyphase interpolation computes output phases directly and avoids zero multiplies.
- Two-channel analysis/synthesis banks split, decimate, interpolate, and recombine subbands.
- Perfect reconstruction requires alias cancellation plus an undistorted desired path.
- Fixed or learned subband front ends provide a natural bridge to lightweight ML.

EXIT QUESTION

What is the key computational idea shared by efficient decimation and efficient interpolation?

Good answer

Use polyphase structure so filtering happens at the low rate and only samples that actually matter are computed.